

GPU Project B:

GPU-Accelerated Partial-Key Brute-Force Attacks

Project Overview

Brute-force key search is the most fundamental attack against symmetric cryptography. In practice, attackers often possess *partial information* about the secret key, reducing the effective search space. In this project, students will perform GPU-accelerated brute-force attacks where a subset of key bits is assumed to be known, and the remaining unknown bits are exhaustively searched. By varying the number of unknown bits, students will experimentally measure attack feasibility and extrapolate results to full-sized cryptographic keys. This project provides concrete intuition for cryptographic security margins and the impact of hardware acceleration.

1 Problem Statement

Let E_k be a block cipher with a secret key $k = (k_{\text{known}}, k_{\text{unknown}})$, where: k_{known} consists of known key bits, and k_{unknown} consists of u unknown key bits. You are given one or more plaintext–ciphertext pairs (P, C) where $C = E_k(P)$. Your task is to recover the unknown portion of the key by exhaustively searching all 2^u possibilities using GPU acceleration.

2 Cryptographic Setup

You can choose block ciphers such as:

- AES-128
- SIMON (any standard variant such as SIMON32/64 or SIMON64/128)
- Any recent stream ciphers

Note: The more ciphers you work on the better. However try to choose ciphers which have similar design principles, such as only feistel ciphers, or spn ciphers or arx based ciphers or stream ciphers. It is your duty to search for ciphers, the best place to search for ciphers is IACR EprintArchive.

Key Partitioning

Let the total key size be n bits. You must fix: $u \in \{1, 2, \dots, 30\}$ unknown key bits and assume the remaining $n - u$ bits are known. You must experiment with multiple values of u .

3 Attack Model

Assume a **known-plaintext attack**: (P, C) such that $C = E_k(P)$.

(Optional but recommended) Use multiple plaintext–ciphertext pairs to eliminate false positives when u is small.

4 CPU Baseline Implementation

Your CPU implementation must:

- Enumerate all 2^u possible values of k_{unknown}
- Construct full candidate keys using k_{known}
- Encrypt the plaintext and test for correctness

Note: Make the code as optimized as possible. Also, describe your methodology for the optimization.

Report:

- Keys tested per second
- Time to recover the correct key

5 GPU Implementation (CUDA)

5.1 Parallelization Strategy

- Each GPU thread can test one candidate value of k_{unknown}
- Thread indices must map directly to candidate keys
- The full key is reconstructed inside the kernel

Note: You could use any other optimization strategy which you have studied or you find in the existing literature. Unique strategies will earn you some bonus points.

5.2 Kernel Design

Each kernel execution must:

- Generate a unique candidate for k_{unknown}
- Combine it with k_{known}
- Encrypt the plaintext
- Compare with the target ciphertext

6 Correctness Verification

You must verify correctness by:

- Testing multiple randomly generated keys
- Verifying recovered keys encrypt correctly
- Reporting failure cases (if any)

Correctness is mandatory for evaluation.

7 Performance Evaluation

You must conduct experiments for: $u = 1, 2, \dots, 30$.

For each value of u , report:

- Total keys searched
- Keys tested per second
- Total time to recover the key

Plot:

- Attack time vs u
- Throughput vs u

8 Extrapolation to Full Key Sizes

Using the measured throughput, estimate the time required to brute-force:

- A 64-bit key
- A 128-bit key

Assume:

- Constant throughput
- Ideal scaling (clearly state limitations)

Discuss why such extrapolations are:

- Useful
- Potentially misleading

Optional Advanced Extensions (Bonus)

- Multi-GPU scaling
- Key bit selection strategies

Deliverables

You must submit:

- CPU and GPU source code
- Performance plots and tables
- A detailed report (6–10 pages)
- Instructions to reproduce results